



理解“缓存流组件”中的书签功能

北京理工大学计算机学院
金旭亮

“书签”功能概述

缓存流组件支持使用`mark()`方法设定一个书签，之后，可以使用`reset()`方法返回那个位置。

▶ `public void mark(int readAheadLimit) throws IOException`

▶ `public void reset() throws IOException`

`mark`方法的`readAheadLimit`参数值比较微妙，让我们通过示例向同学们作介绍。

准备要测试的数据



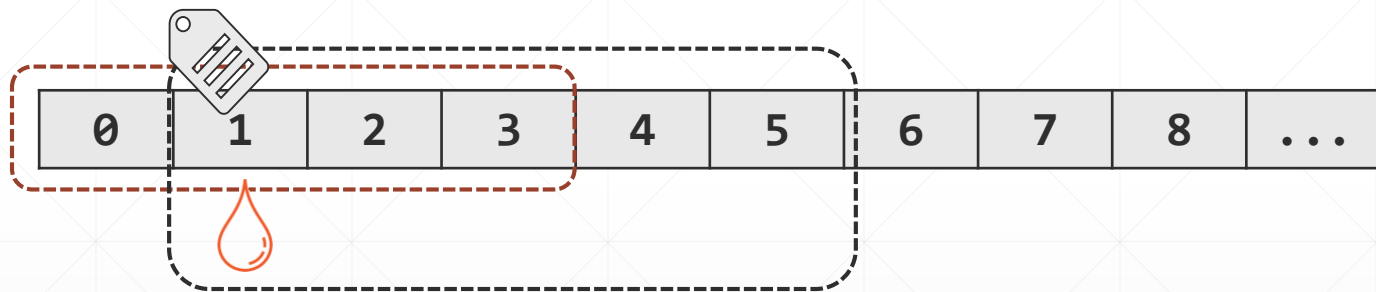
```
private static byte[] initBuffer() {  
    byte[] buff = new byte[127];  
    for (int i = 0; i < buff.length; i++) {  
        buff[i] = (byte) i;  
    }  
    return buff;  
}
```



0	1	2	3	4	5	6	7	8	...
---	---	---	---	---	---	---	---	---	-----

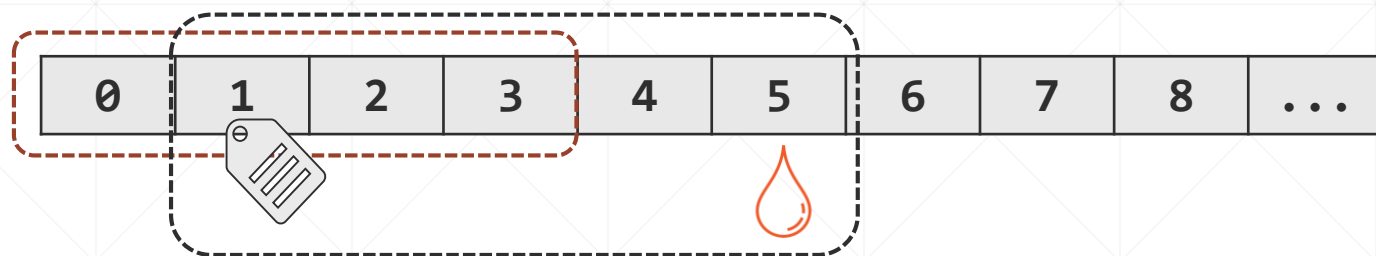
理解书签功能

```
//缓冲区大小设置为4个字节  
var bufferedReader = new BufferedReader(reader, 4);  
  
bufferedReader.read(); //读入0  
bufferedReader.mark(5);
```

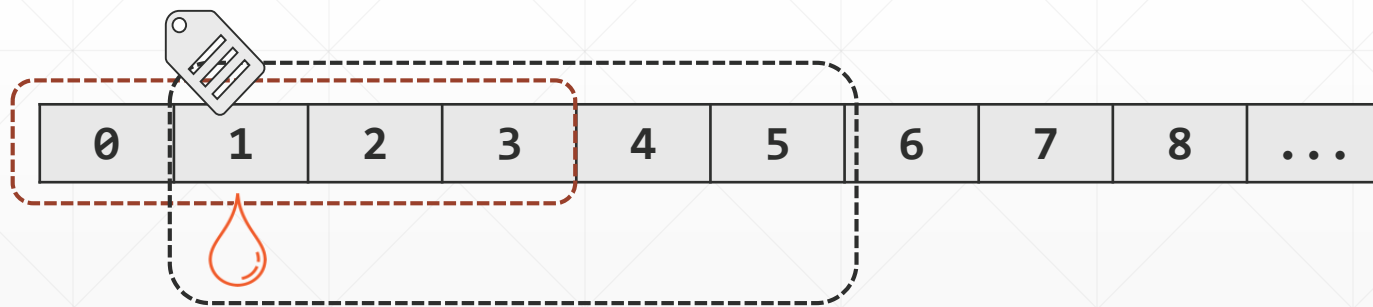


mark方法给“1”打上书签，由于其参数指定为5，所以，并从这里往后再读5个字节，书签应该有效，这时，缓存区实际上可以放5个字符。

重复5次 `bufferedReader.read()`



在这里reset, 没问题, 因为这时“1”还在缓冲区中, 因此, reset之后.....



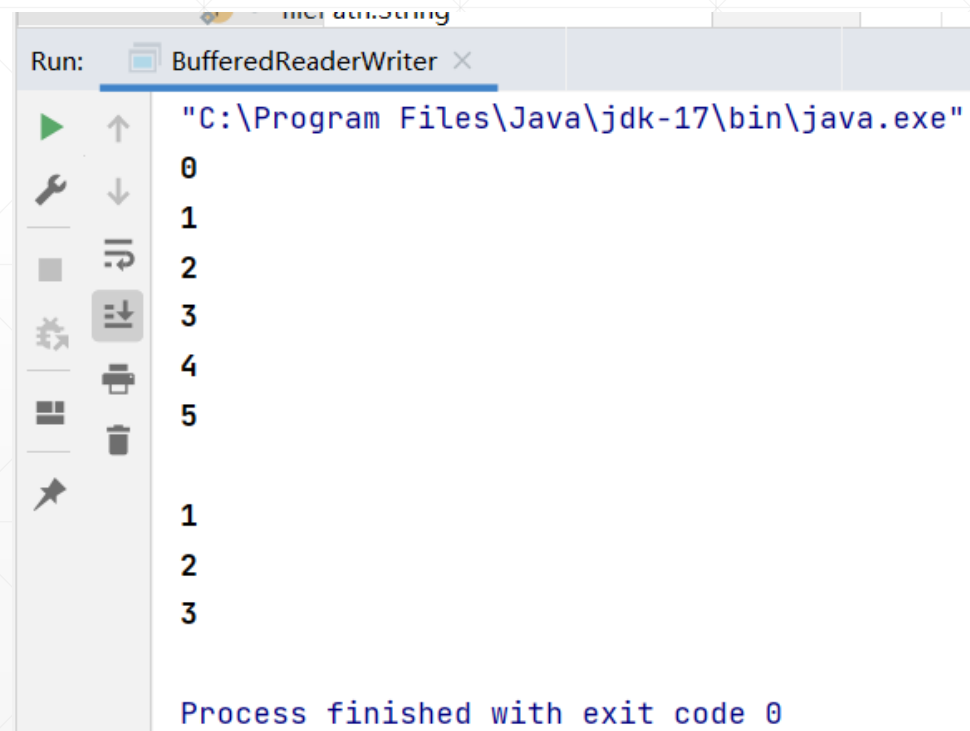
这时的输出, 就是从1开始的。

```

private static void readFromFileUseMarkSuccess(byte[] buff) {
    try (
        var inputStream = new ByteArrayInputStream(buff);
        var reader = new InputStreamReader(inputStream);
        //缓冲区大小设置为4个字节
        var bufferedReader = new BufferedReader(reader, 4);
    ) {
        System.out.println(bufferedReader.read()); //0
        //给“1”打上书签，并设置从这里往后再读5个字节，这时，缓冲区实际上可以放5个数字
        bufferedReader.mark(readAheadLimit: 5);
        System.out.println(bufferedReader.read()); //1
        System.out.println(bufferedReader.read()); //2
        System.out.println(bufferedReader.read()); //3
        System.out.println(bufferedReader.read()); //4
        System.out.println(bufferedReader.read()); //5
        //在这里reset,没问题，因为这时“1”还在缓冲区中
        bufferedReader.reset();
        System.out.println();
        System.out.println(bufferedReader.read()); //1
        System.out.println(bufferedReader.read()); //2
        System.out.println(bufferedReader.read()); //3
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

一切“正常”情况下的程序输出：



```

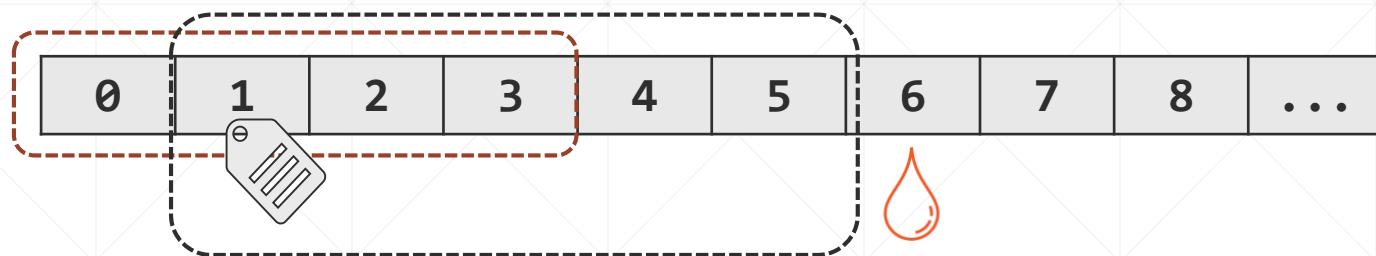
Run: BufferedReaderWriter x
"C:\Program Files\Java\jdk-17\bin\java.exe"
0
1
2
3
4
5

1
2
3

Process finished with exit code 0

```

如果重复6次 `bufferedReader.read()`



在这里reset，出事了，因为这时“1”已不在缓冲区中，因此，reset之后.....



程序会抛出异常.....

```
private static void readFromFileUseMarkFailure(byte[] buff) {
    try (
        var inputStream = new ByteArrayInputStream(buff);
        var reader = new InputStreamReader(inputStream);
        //缓冲区大小设置为4个字节
        var bufferedReader = new BufferedReader(reader, 4);
    ) {
        System.out.println(bufferedReader.read()); //0
        //给"1"打上书签, 并设置从这里往后再读5个字节, 书签应该有效, 这时, 缓冲区实际上可以放5个数字
        bufferedReader.mark(5);
        System.out.println(bufferedReader.read()); //1
        System.out.println(bufferedReader.read()); //2
        System.out.println(bufferedReader.read()); //3
        System.out.println(bufferedReader.read()); //4
        System.out.println(bufferedReader.read()); //5
        //读入5以后, 继续往后读, 这时"1"已不在缓冲区
        System.out.println(bufferedReader.read()); //6
        //在这里reset, 会失败, 因为"1"已不在缓冲区
        //如果将前面readAheadLimit设置为6, 则reset又可以成功了, 因为"1"这时还在缓冲区中
        bufferedReader.reset();
        System.out.println();
        System.out.println(bufferedReader.read());
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```



书签无效, 此时reset()导致异常抛出!

```
5
6
java.io.IOException Create breakpoint : Mark invalid
    at java.base/java.io.BufferedReader.reset(BufferedReader.java:514)
    at BufferedReaderWriter.readFromFileUseMarkFailure(BufferedReaderWriter.java:98)
    at BufferedReaderWriter.readFromFileUseMark(BufferedReaderWriter.java:49)
    at BufferedReaderWriter.main(BufferedReaderWriter.java:12)
```


“书签”功能小结



只要mark书签还在缓存区范围之内，reset总是可以成功的！



ReadAheadLimit的意思是：你要设置了这个值，那我保证在读入这么多的数据之前，你的mark一定是有效的，如果你的值比较大，那我就扩充缓存区！



当程序要读取的数据比较多，超过了ReadAheadLimit的限度，也就是说，大于缓存区从“mark位置 + 缓冲区大小”的值，这意味着mark值已不在缓存区中，这时，reset就失败了！

“书签”功能的一个典型应用场景



在解析HTML网页时，可以使用mark“记住”开始标记（比如<div>）的位置，当发现结束标记（比如</div>）时，就可以使用书签“回到”开头，从而提取出整个div元素的内容。